III Quantum Computing and Quantum Algorithms

1. The circuit model

a) Classical computation

Use of classical computers (abstractly):

Solve problems $\equiv$ compute functions

$$f : \{0,1\}^n \longrightarrow \{0,1\}^m$$

$$\underline{x} = (x_1, \dots, x_n) \longmapsto f(x_1, \dots, x_n)$$

The function f depends on the problem we want to solve, $\underline{x}$ encodes the instance of the problem.

E.g.: Problem = multiplication : $(a, b) \longmapsto a \cdot b$

$$\underline{x} = (\underline{x}^1, \underline{x}^2) \longmapsto f(\underline{x}) = \underline{x}^1 \cdot \underline{x}^2$$

encoded in binary

Problem = Factorisation:

$\underline{x}$: integer; $f(\underline{x})$: list of prime factors (suitably encoded)

More precisely:

Each problem is encoded by a family of functions $f \equiv f^{(n)} : \{0,1\}^{m} \to \{0,1\}^{m'}$, with $m = \text{poly}(n)$, $n \in \mathbb{N}$ – one for each input size.

i.e.: m grows at most polynomially with n (technically, $\exists \alpha > 0$ s.th. $\frac{m}{n^{\alpha}} \to 0$).

(Technical point: It must be possible to "construct the functions $f^{(n)}$ systematically and efficiently", see later!)

Which ingredients do we need to compute a general function f?

(i) $f : \{0,1\}^n \longrightarrow \{0,1\}^m$

$$f(\underline{x}) = (f_1(\underline{x}), f_2(\underline{x}), \ldots, f_m(\underline{x}))$$

where $f_k(\underline{x}) : \{0,1\}^n \longrightarrow \{0,1\}$

$\Rightarrow$ can restrict analysis to boolean functions

$$f : \{0,1\}^n \longrightarrow \{0,1\}.$$

(ii) Define $L = \{\underline{y} \mid f(\underline{y}) = 1\} = \{\underline{y}^1, \underline{y}^2, \ldots, \underline{y}^\ell\}$.

Define $\delta_{\underline{y}}(\underline{x}) = \begin{cases} 0 ; & \underline{x} \neq \underline{y} \\ 1 ; & \underline{x} = \underline{y} \end{cases}$ ← bitwise equality!

Then, $f(\underline{x}) = \delta_{\underline{y}^1}(\underline{x}) \vee \delta_{\underline{y}^2}(\underline{x}) \vee \ldots \vee \delta_{\underline{y}^\ell}(\underline{x})$

"$\vee$": logical "or":

$0 \vee 0 = 0$

$0 \vee 1 = 1$

$1 \vee 0 = 1$

$1 \vee 1 = 1$

($0 \equiv$ "false", $1 \equiv$ "true")

"$\vee$" is associative:

$$a \vee b \vee c := (a \vee b) \vee c = a \vee (b \vee c)$$

and commutative: $a \vee b = b \vee a$.

(iii) Define <u>Kronecker</u> δ:

$$\delta_y(x) = \begin{cases} 0 : & y \neq x \\ 1 : & y = x \end{cases}$$

Then,

$$\underline{\delta}_{\underline{y}}(\underline{x}) = \delta_{y_1}(x_1) \wedge \delta_{y_2}(x_2) \wedge \dots \wedge \delta_{y_n}(x_n)$$

"$\wedge$": logical "and":

$0 \wedge 0 = 0$

$0 \wedge 1 = 0$

$1 \wedge 0 = 0$

$1 \wedge 1 = 1$

($0 \equiv$ "false", $1 \equiv$ "true")

"$\wedge$" is <u>associative</u> & <u>commutative</u>;

"$\wedge$" & "$\vee$" are distributive:

$$(a \vee b) \wedge c = (a \wedge c) \vee (b \wedge c).$$

(In essence, same rules as $\wedge \to \cdot$, $\vee \to +$)

(iv)

$$\delta_y(x) = \begin{cases} x & \text{if } y = 1 \\ \neg x & \text{if } y = 0 \end{cases}$$



Logical "not": $\neg 0 = 1$

$\neg 1 = 0$

Combine (i) – (iv):

Any $f(\underline{x})$ can be constructed from 4 ingredients:

"and", "or", "not" gates,

plus a "copy" gate $x \longmapsto (x, x)$.

This is called a universal gate set.

(Note: In fact, already either $\neg(x \wedge y)$ "nand", or $\neg(x \vee y)$ "nor" are universal, together with "copy".)

This gives rise to the

Circuit model of computation:

The functions $f \equiv f^{(n)}$ which we can compute are constructed by concatenating gates from a

simple universal gate set (e.g. and/or/not/copy) sequentially in time (i.e., there are no loops allowed). This gives rise to a circuit for $f^{(n)}$.

The difficulty ("computational hardness") of a problem in the circuit model is measured by the number $K(n)$ of elementary gates needed to compute $f^{(n)}$ ($\hat{=}$ # of time steps).

We often distinguish two qualitatively different regimes:

$K(n) \sim \text{poly}(n)$: efficiently solvable (class P)
easy problem

$K(n) \gg \text{poly}(n)$ — e.g. $K(n) \sim \exp(n^\alpha)$:
hard problem

(Technical note: We must impose that the circuits

used for $f^{(n)}$ are uniform, i.e. they can be generated efficiently – e.g. by a simple n-independent computer program. More formally, $f^{(n)}$ should be generated by a Turing machine.)

Example:

f = Multiplication:

Efficient:

$$\overbrace{10110}^{\ell} \times \overbrace{10011}^{\ell'}$$

$$\begin{array}{r} 10110 \\ 10110 \\ 10110 \\ \hline 110100010 \end{array} \quad \Big\} \ell'$$

$\ell \times \ell'$ additions:

$O(\ell\ell') \sim O(n^2)$ gates.

f: Factorization.

E.g.: Sieve of Eratosthenes:

$\{0,1\}^n \to$ try about $\sqrt{2^n} \sim 2^{n/2}$ cases

$\Longrightarrow$ hard / exp. scaling.

No efficient algorithm known!

Is a typical problem easy or hard?

$f : \{0,1\}^n \longrightarrow \{0,1\}$

of different f: $2^{(2^n)}$

(2^n: # of inputs; $f(x) = \{0, 1$ for each input)

But: there are only $c^{poly(n)}$ circuits of length $poly(n)$!

($poly(n)$: # of elem. gates)

$\Rightarrow$ As n gets large, most f cannot be computed efficiently (i.e. with $poly(n)$ operations).

Does the computational power depend on the gate set?

NO! By definition, any universal gate set can simulate any other gate set with constant overhead!

Remark: There is a wide range of alternative models of computation, some more and more less realistic:

- CPU
- parallel computers
- "Turing machines" — tape + read/write head
- cellular automata
- ... and lots of exotic models ...

But: All known "reasonable" models of computation can simulate each other with poly(n) overhead $\Rightarrow$ same computational power (in the sense above).

Church-Turing Thesis: All reasonable models of computation have the same computational power.

5) Reversible circuits

For quantum computing – coming soon – we will use the circuit model.

Gates will be replaced by unitaries.

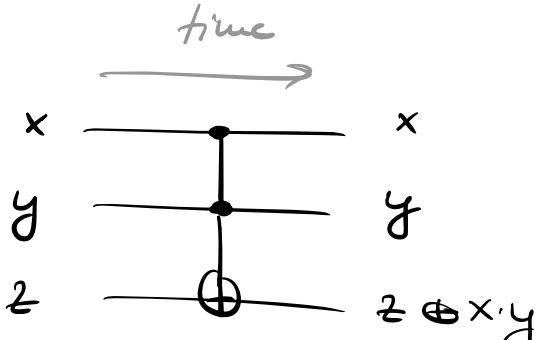
But: Unitaries are reversible,

while classical gates (and, or) are irreversible.

Could such a model even do classical computations – i.e., can we find a universal gate set with only reversible gates?

YES! – Classical computation can be made reversible:

Toffoli gate:



"XOR" = addition mod 2:

$0 \oplus 0 = 0$
$0 \oplus 1 = 1$
$1 \oplus 0 = 1$
$1 \oplus 1 = 0$

also assoc., comm., & distr. w/ $\wedge$. (like "$\vee$")

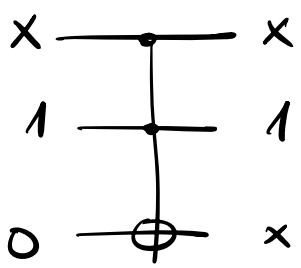
→ Toffoli gate is reversible

(it is its own inverse, since $(z \oplus x \cdot y) \oplus x \cdot y = z$)

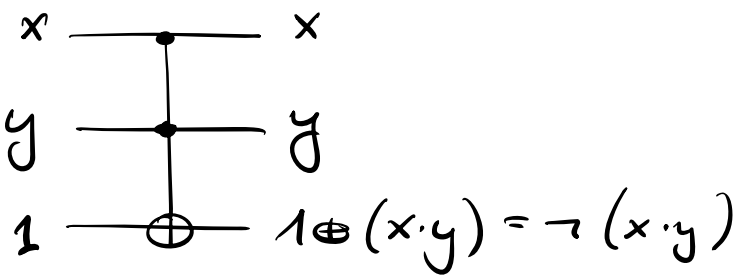
→ Toffoli gate can simulate and/or/not/copy,

by using ancillas in state "0" or "1":

E.g.:



"copy"



"Nand"

⟹ gives reversible universal gate set

(but requires ancillas)

Thus can be used to compute any $f(\underline{x})$ reversibly,

using ancillas, with essentially the same # of gates:

$$f^R(\underline{x}, y) \longmapsto (\underline{x}, f(\underline{x}) \oplus y)$$

↑ bitwise XOR.

(Idea: Replace any gate by a reversible gate using ancillas. Then XOR the result into the Y register. Finally, run the circuit backwards to "uncompute" the ancillas. Ancilla count can be optimized for → cf. Preskill's notes.)

⟹ Everything can be computed reversibly.

BUT: 3-bit gate is required! (→ Homework)

c) Quantum Circuits

Most common model for quantum computation:

The circuit model:

- Quantum system consisting of qubits: tensor product structure.
- Universal gate set $S = \{U_1, \dots, U_k\}$ of few-qubit gates (typ. 1- and 2-qubit gates) U_j. (See later for definition of "universal"!)
- Construct circuits by sequentially applying

elements of S to a subset of qubits:

$$|\psi_{out}\rangle = V_T V_{T-1} \cdots V_1 |\psi_{in}\rangle$$

V_j acting on subset of qubits

- Initial state:

$$|\psi_{in}\rangle = |x_1\rangle |x_2\rangle \cdots |x_n\rangle \overbrace{|0\rangle |0\rangle \cdots |0\rangle}^{\ell}$$

$$= |\underline{x}\rangle |\underline{0}\rangle$$

$\underline{x}$: encodes instance of problem

$\underline{0}$: ancillas

– alternatively, we can also have

$$|\psi_{in}\rangle = |\underline{0}\rangle \equiv |0\rangle^{\otimes \ell}$$

and encode the instance in the circuit.

- At the end of the computation, measure the final state $|\psi_{out}\rangle$ in the computational basis $\{|0\rangle, |1\rangle\}$

⟶ outcome $|y\rangle$ w/ prob. $p(y) = |\langle y|\psi_{out}\rangle|^2$

Notes: • This is a probabilistic scheme — it outputs y w/ some prob. $p(y)$. In principle, we should compare to class. probabilistic schemes — see later.

• We need not measure all qubits —

not measuring = tracing = measuring and ignoring outcome

• POVMs don't help — we can simulate them ($\Rightarrow$ Naimark). Similarly, CP maps don't help — we can simulate them (Stinespring + trace ancilla).

• Measurements at earlier times don't help: Can always postpone them (they commute). If gate at later time would depend on meas. outcome: This dependence can be realized inside the circuit w/ "controlled gates"

(cf. later + homework)

What gate set should we choose?

- There is a continuum of gates — situation much more rich.

- Different notions of universality exist:

  - exact universality: Any n-qubit gate can be realized exactly.

    → Requires a continuous family of universal gates (counting argument!)

  - approximate universality: Any n-qubit gate can be approximated well by gate set

    (Finite gate set sufficient;

    Solovay-Kitaev-Theorem: ε-approximation (in $\|\cdot\|_\infty$-Norm) of 1-qubit gate requires $O(\text{poly}(\log(1/\varepsilon)))$ gates from a suitable finite set.)

- 1- and 2-qubit gates alone are universal!

  (cf. classical: 3-bit gates needed!!)

- For approximate universality, almost any gate
  two-qubit gate will do! ↑ w/ prob. 1.
- More univ. sets: later!

d) Universal gate set

Our exact universal gate set:

(i) 1-qubit rotations about X & Z axis:

$$R_x(\phi) = e^{-iX\phi/2} \quad ; \quad X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \; X^2 = I$$

$$R_z(\phi) = e^{-iZ\phi/2} \quad ; \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}, \; Z^2 = I.$$

For $\Pi^2 = I$: $e^{-i\Pi\phi/2} = \cos\phi/2 \, I - i \sin\phi/2 \, \Pi$

$$\Rightarrow R_x(\phi) = \begin{pmatrix} \cos\phi/2 & -i\sin\phi/2 \\ -i\sin\phi/2 & \cos\phi/2 \end{pmatrix}$$

$$R_z(\phi) = \begin{pmatrix} e^{-i\phi/2} & 0 \\ 0 & e^{i\phi/2} \end{pmatrix}$$

Can be understood as rotations on Bloch sphere about X/Z axis by angle ϕ (i.e., rotations in $SO(3) \cong SU(2)/\mathbb{Z}_2$).

Together, R_x and R_z generate all rotations in $SO(3)$ (Euler angles!), and thus in $SU(2)$ up to a phase.

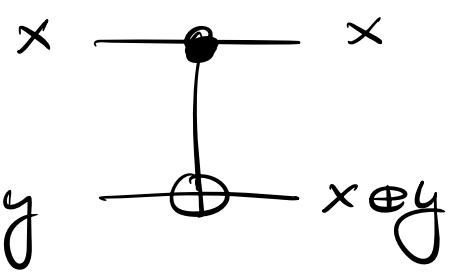
Lemma: For any $U \in SU(2)$,

$$U = e^{i\phi} R_x(\alpha) R_z(\beta) R_x(\gamma) \quad \text{for some } \phi, \alpha, \beta, \gamma.$$

Proof: Homework.

(ii) one two qubit gate (almost all would do!).

Typically, we use "controlled-NOT" = "CNOT":

CNOT = 

$$= \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

CNOT flips y iff $x = 1$: classical gate!

Can prove: This gate set can create any n-qubit U exactly (but of course not efficiently – U has $\sim (2^n)^2 = 4^n$ real parameters).

Overview of a number of important gates & identities

(Proof/check: Homework!)

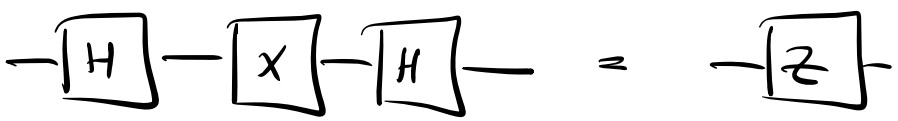
Hadamard gate: $H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$

$$H = H^\dagger; \quad H^2 = I.$$

$$H R_x(\phi) H = R_z(\phi)$$

$$H R_z(\phi) H = R_x(\phi)$$

Graphical "circuit" notation:

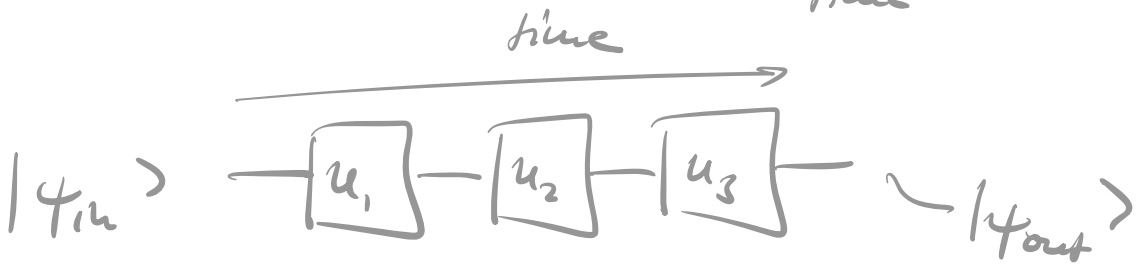


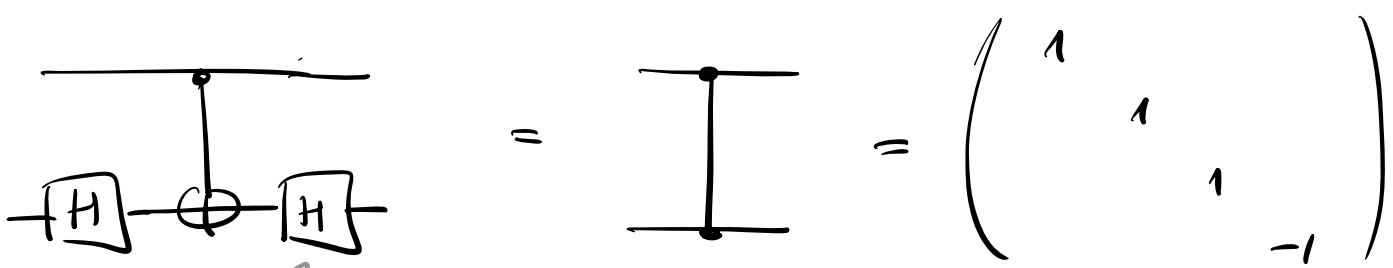
Important:

Matrix notation: time goes right to left

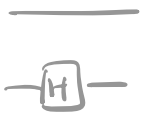
Circuit notation: time goes left to right:

i.e.: $|\psi_{in}\rangle \longmapsto |\psi_{out}\rangle = \underbrace{U_3 U_2 U_1}_{\text{time}} |\psi_{in}\rangle$





only applied to 2nd qubit, i.e:

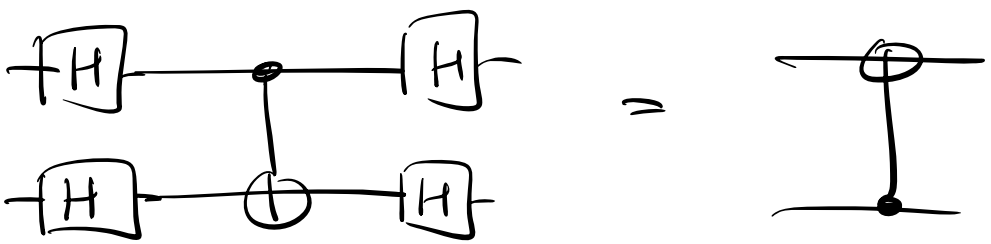


$\equiv I \otimes H$.

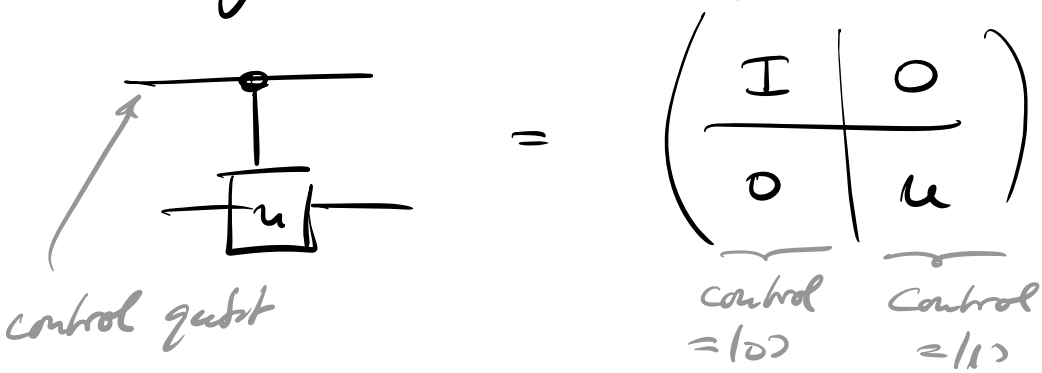
"Controlled-Z"

"Controlled-Phase"

CZ, CPHASE

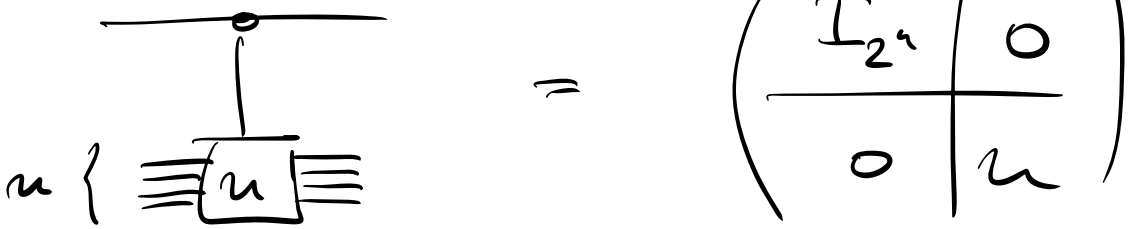


Generally: For a unitary $U \in SU(2)$,

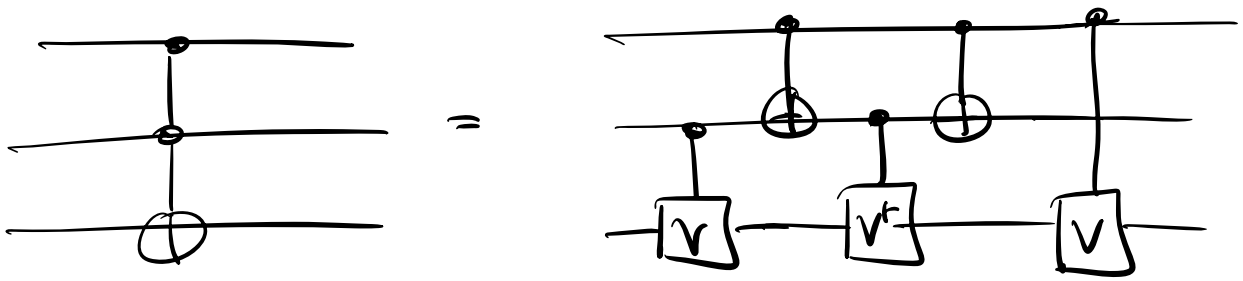


Can be implemented w/ 2 CNOT ($\to$ HW!)

Also for $U \in SU(2^n)$:



Circuit for Toffoli:



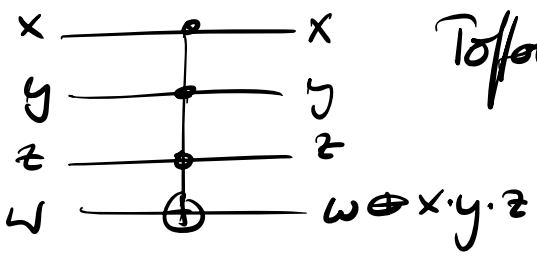
with $V = \frac{1-i}{2}(I + iX)$

U to controlled-U:

Given circuit for U – in particular, a classical reversible circuit — we can also build controlled-U:

Just replace every gate by its controlled-version,

in particular Toffoli by



Toffoli w/ 3 controls can be built from normal Toffoli (since class. universal!)

Finally, some further approx. universal gate sets:

- CNOT + 2 random 1-qubit gates
- CNOT + H + $T = R_z(\pi/4)$ ("$\pi/8$ gate")

2. Oracle-based algorithms

a) The Deutsch algorithm

Consider $f: \{0,1\} \longrightarrow \{0,1\}$

Let f be "very hard to compute" – e.g. long circuit

Want to know: Is $f(0) = f(1)$?

(e.g.: will a specific chess move affect result?)

How often do we have to run the circuit for f

(= "evaluate f")? – We think of f as a "black box"

or "oracle": How many oracle queries are needed?

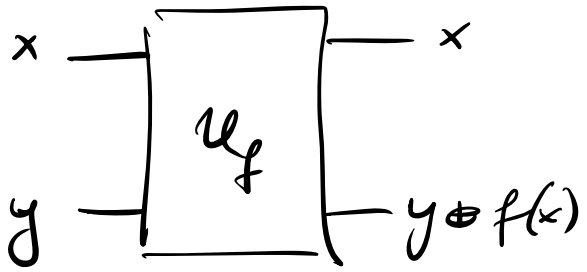
Classically, we clearly need 2 queries:

Compute $f(0)$ and $f(1)$.

Can quantum physics help?

Consider reversible implementation of f:

$$f^R: (x, y) \longmapsto (x, y \oplus f(x))$$

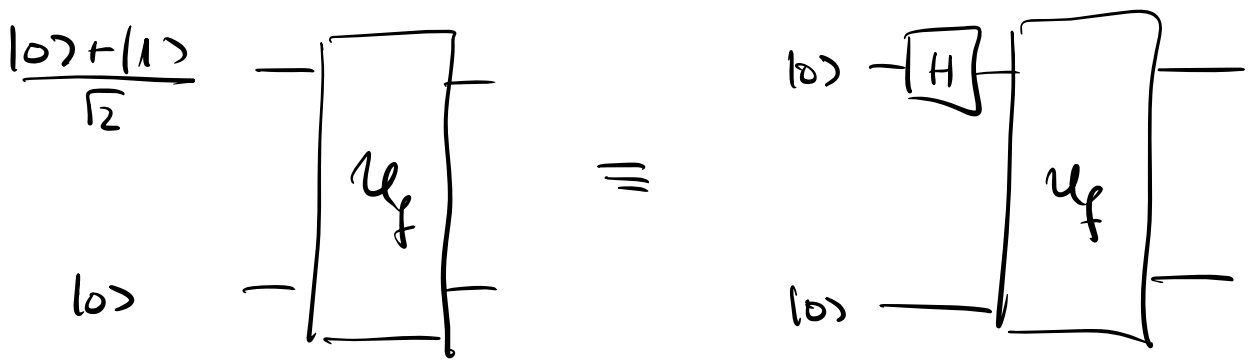


$$|x\rangle|y\rangle \longmapsto |x\rangle|y \oplus f(x)\rangle$$

Of course, we can use U_f to compute $f(0)$ or $f(1)$ on a quantum computer, but these we could also do classically. So, can we do better than this?

Try to use <u>superpositions as inputs</u>?

<u>first attempt:</u>



$$\frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes |0\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle|0\rangle + |1\rangle|0\rangle\right) \xrightarrow{U_f} \frac{1}{\sqrt{2}}\left(|0\rangle|f(0)\rangle + |1\rangle|f(1)\rangle\right)$$

$\longrightarrow$ have evaluated f on <u>both outputs!</u>

But how can we extract the relevant information from this state (i.e. do a measurement)?

- Meas. in comp. basis: collapse superpos. to one case!
- More generally: If $f(0) \neq f(1)$, the output is in

$$S_{\neq} = \left\{ \frac{1}{\sqrt{2}} (|0\rangle|0\rangle + |1\rangle|1\rangle), \frac{1}{\sqrt{2}} (|0\rangle|1\rangle + |1\rangle|0\rangle) \right\},$$

and for $f(0) = f(1)$ in

$$S_{=} = \{ |+\rangle|0\rangle, |+\rangle|1\rangle \}.$$

$\Rightarrow$ not orthogonal, i.e. not (deterministically) distinguishable!

But: We can do measurements which, with some probability, allow to conclude that $f(0) = f(1)$ or $f(0) \neq f(1)$.

E.g., all states in $S_{=}$ are orth. to

$R_{\neq} = \{ |+\rangle|0\rangle, |+\rangle|1\rangle \}$, and all states in $S_{\neq}$ to $R_{=} = \left\{ \frac{1}{\sqrt{2}}(|0\rangle|0\rangle - |1\rangle|1\rangle), \frac{1}{\sqrt{2}} (|0\rangle|1\rangle - |1\rangle|0\rangle) \right\}$.

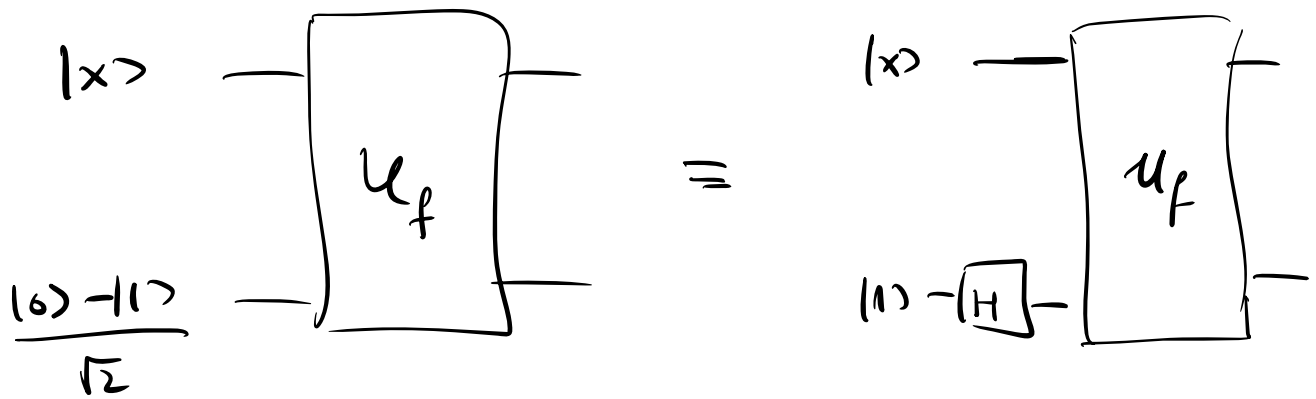
$\Rightarrow$ A POVM which includes those outcomes plus an extra "fail" outcome allows to

unambiguously identify whether $f(0) = f(1)$?, with some probability

Optimal success probability: $\frac{1}{2}$ ($\rightarrow$ Homework)

While that is impossible classically, it does not give an improvement on average.

Second attempt:



$$|x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right) \xrightarrow{U_f} |x\rangle \left(\frac{|f(x)\rangle - |1 \oplus f(x)\rangle}{\sqrt{2}} \right) =$$

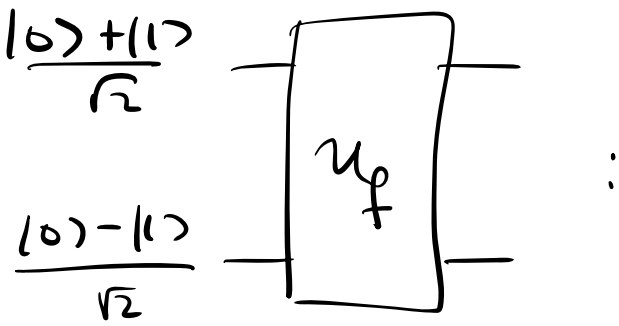
$$= \left\{ \begin{array}{ll} f(x) = 0: & |x\rangle \; \frac{|0\rangle - |1\rangle}{\sqrt{2}} \\ f(x) = 1: & |x\rangle \; \frac{|1\rangle - |0\rangle}{\sqrt{2}} \end{array} \right\}$$

$$= |x\rangle \left[(-1)^{f(x)} \; \frac{|0\rangle - |1\rangle}{\sqrt{2}} \right]$$

$$= (-1)^{f(x)} |x\rangle \left(\frac{|0\rangle - |1\rangle}{\sqrt{2}} \right)$$

Not useful by itself: $f(x)$ only encoded in global phase for each classical input $|x\rangle$.

Can do one step:



$$\frac{|0\rangle+|1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle-|1\rangle}{\sqrt{2}} = \frac{1}{\sqrt{2}} \left(|0\rangle \frac{|0\rangle-|1\rangle}{\sqrt{2}} + |1\rangle \frac{|0\rangle-|1\rangle}{\sqrt{2}} \right)$$

$$= \frac{1}{\sqrt{2}} \left((-1)^{f(0)} |0\rangle \frac{|0\rangle-|1\rangle}{\sqrt{2}} + (-1)^{f(1)} |1\rangle \frac{|0\rangle-|1\rangle}{\sqrt{2}} \right)$$

$$= \frac{(-1)^{f(0)}|0\rangle + (-1)^{f(1)}|1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle-|1\rangle}{\sqrt{2}}$$

Observations:

→ No entanglement created (!)

→ 2nd qubit – the one where U_f outputs the function value – is unchanged (!!!)

→ 1st qubit gets a phase $(-1)^{f(x)}$

("phase kick-back technique")

State of 1st qubit:

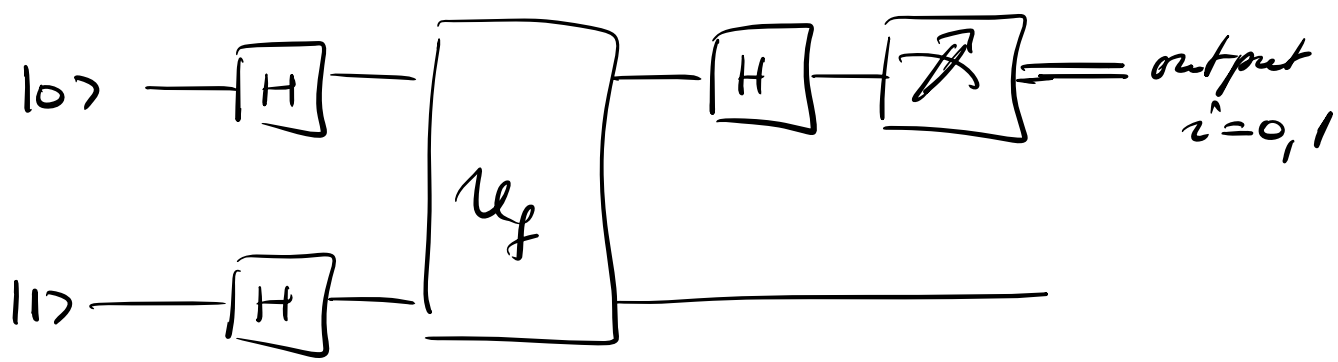
$f(0) = f(1) \iff \frac{|0\rangle + |1\rangle}{\sqrt{2}}$

$f(0) \neq f(1) \iff \frac{|0\rangle - |1\rangle}{\sqrt{2}}$

(up to irrelevant global phase)

Orthogonal states! $\Longrightarrow$ measurement of 1st qubit in basis $\{|+\rangle, |-\rangle\}$ (or apply $-H-$ & measure in $\{|0\rangle, |1\rangle\}$) allows to decide if $f(0) \stackrel{?}{=} f(1)$!

Deutsch algorithm:



output $i=0$: $\Rightarrow f(0) = f(1)$

$i=1$: $\Rightarrow f(0) \neq f(1)$

One application of U_f has been sufficient!

$\Rightarrow$ Speed-up compared to class. algorithm

(1 vs. 2 oracle queries).

Interesting to note: 2nd qubit never needs to be measured – and it contains no information.

Two main insights:

- Use input $\sum |x\rangle$ to evaluate f on all inputs simultaneously.
- This parallelism alone is not enough – need a smart way to read out the relevant information.

However, a constant speed-up is not that impressive – in particular, it is highly architecture-dependent!

Thus:

b) The Deutsch-Jozsa algorithm

Consider $f: \{0,1\}^n \to \{0,1\}$ with promise (i.e., a condition we know is met by f) that

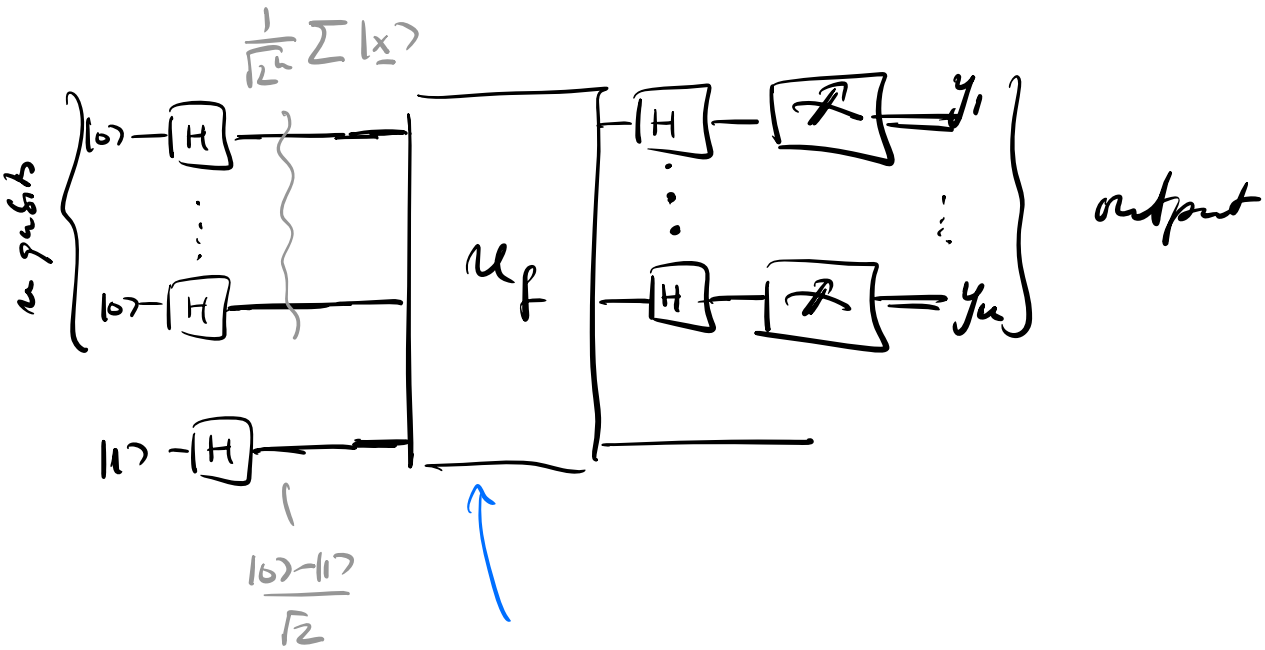
either $f(x) = c \quad \forall x$ ("f constant")

or $|\{x \mid f(x)=0\}| = |\{x \mid f(x)=1\}|$ ("f balanced")

Want to know: Is f constant or balanced?

How many queries needed?

Use same idea: Input $\sum |x\rangle$ and $\frac{|0\rangle - |1\rangle}{\sqrt{2}}$:



$U_f: |x\rangle|y\rangle \mapsto |x\rangle|y \oplus f(x)\rangle$

Before analysing circuit: What is action of H?

$$H: |x\rangle \longmapsto \frac{1}{\sqrt{2}} \sum_{y=0,1} (-1)^{x\cdot y} |y\rangle$$

$$H^{\otimes n}: |x_1,\dots,x_n\rangle \longmapsto \frac{1}{\sqrt{2^n}} \sum_{\underline{y}} (-1)^{x_1 y_1} \cdot \dots \cdot (-1)^{x_n y_n} |y_1,\dots,y_n\rangle$$

or:

$$|\underline{x}\rangle \longmapsto \frac{1}{\sqrt{2^n}} \sum (-1)^{\underline{x}\cdot \underline{y}} |\underline{y}\rangle$$

where $\underline{x}\cdot\underline{y} := x_1 y_1 \oplus x_2 y_2 \oplus \dots \oplus x_n y_n$

("scalar product" mod 2).

↖ is NOT a scalar product!

Analysis of circuit:

$$|0\rangle|1\rangle \xrightarrow{H^{\otimes n}\otimes H} \left(\sum_{\underline{x}} |\underline{x}\rangle\right)(|0\rangle - |1\rangle)$$

we omit normalization!

phase kick-back

$$\xrightarrow{U_f} \left(\sum_{\underline{x}} (-1)^{f(\underline{x})} |\underline{x}\rangle\right)(|0\rangle - |1\rangle)$$

$$\xrightarrow{H^{\otimes n}\otimes I} \left(\sum_{\underline{y}} \underbrace{\sum_{\underline{x}} (-1)^{f(\underline{x})+\underline{x}\cdot\underline{y}}}_{=: a_{\underline{y}}} |\underline{y}\rangle\right)(|0\rangle - |1\rangle)$$

$P_y := |a_y|^2$ is the probability to measure $y = (y_1, \dots, y_n)$.

f constant: $f(x) = c$

$$a_y = (-1)^c \underbrace{\sum_x (-1)^{x \cdot y}}_{\propto \delta_{y,0}} = (-1)^c \delta_{y,0}$$

f balanced:

For $y = 0$: $a_0 = \sum_x (-1)^{f(x) + x \cdot 0}$

$$= \sum_x (-1)^{f(x)} = 0$$

↑ f balanced!

Thus:

Output $y = 0$ ⟹ f constant

Output $y \neq 0$ ⟹ f balanced

⟹ We can unambiguously distinguish the 2 cases with one query to the oracle for f!

What is the speed-up vs. classical methods?

Quantum: 1 use of f.

Classical: Worst case, we have to determine $2^{n-1}+1$ values of f to be sure!

$\Rightarrow$ exponential vs. constant!

But: If we are ok to get right answer with very high probability $p = 1 - p_{error}$, then for k queries to f,

$$p_{error} \approx 2 \cdot \underbrace{\left(\frac{1}{2}\right)^k}_{\approx \text{prob. to get } k\times \text{ same outcome}}$$

for balanced f, if $k \ll 2^n$.

i.e.: $k \sim \log(1/p_{error})$.

Randomized classical: Much smaller speed-up vs. randomized classical algorithm (even for exp. small error, $k \sim n$ oracle calls are sufficient.)

c) Simon's algorithm

... will give us a true exponential speedup (also rel. to randomized class. algorithms) in terms of oracle queries!

Oracle: $f : \{0,1\}^n \to \{0,1\}^n$

with promise:

$\exists\, \underline{a} \neq \underline{0}$ s.t. $f(\underline{x}) = f(\underline{y})$ exactly if $\underline{y} = \underline{x} \oplus \underline{a}$.

("hidden periodicity")

Task: Find $\underline{a}$ by querying f.

Classical: Need to query $f(\underline{x}_i)$ until pair $\underline{x}_i, \underline{x}_j$ with $f(\underline{x}_i) = f(\underline{x}_j)$ is found.

Roughly: k queries $x_1, \dots, x_k \longrightarrow \sim k^2$ pairs,

for each pair: prob $(f(\underline{x}_i) = f(\underline{x}_j)) \approx 2^{-n}$

$\Rightarrow$ $p_{\text{success}} \sim k^2 2^{-n}$

$\Rightarrow$ need $k \sim 2^{cn}$ queries!

Quantum algorithm (Simon's algorithm):

i) Start with $\frac{1}{\sqrt{2^n}} \sum_{\underline{x}} |\underline{x}\rangle = H^{\otimes n} |\underline{0}\rangle$

ii) Apply $U_f : |\underline{x}\rangle|y\rangle \longmapsto |\underline{x}\rangle|y \oplus f(\underline{x})\rangle$

$$U_f : \left(\frac{1}{\sqrt{2^n}} \sum_{\underline{x}} |\underline{x}\rangle_A \right) |\underline{0}\rangle_B \longmapsto \frac{1}{\sqrt{2^n}} \sum_{\underline{x}} |\underline{x}\rangle_A |f(\underline{x})\rangle_B$$

iii) Measure B. $\Rightarrow$ Collapse onto random $f(\underline{x}_0)$
(and thus random $\underline{x}_0$).

$\Rightarrow$ Register A collapses onto

$$\frac{1}{N} \sum_{\underline{x} : f(\underline{x}) = f(\underline{x}_0)} |\underline{x}\rangle = \frac{1}{\sqrt{2}} \left(|\underline{x}_0\rangle + |\underline{x}_0 \oplus \underline{a}\rangle \right)$$

— How can we extract $\underline{a}$? —

(Meas. in comp. basis $\rightarrow$ collapse on rand. $\underline{x}_0$: useless.)

iv) Apply $H^{\otimes n}$ again:

$$H^{\otimes n} \left(\frac{1}{\sqrt{2}} \left(|\underline{x}_0\rangle \oplus |\underline{x}_0 \oplus \underline{a}\rangle \right) \right)$$

$$H^{\otimes n} |\underline{x}\rangle \propto \sum_{\underline{y}} (-1)^{\underline{x} \cdot \underline{y}} |\underline{y}\rangle$$

$$= \frac{1}{\sqrt{2^{n+1}}} \sum_y \left[(-1)^{x_0 \cdot y} + (-1)^{(x_0 + a) \cdot y} \right] |y\rangle$$

$$\hookrightarrow a \cdot y = 0 \Rightarrow = 2 \cdot (-1)^{x_0 \cdot y}$$

$$a \cdot y = 1 \Rightarrow = 0$$

$$= \frac{1}{\sqrt{2^{n-1}}} \sum_{y : a \cdot y = 0} (-1)^{x_0 \cdot y} |y\rangle$$

v) Measure in comp. basis:

$\Rightarrow$ obtain random y s.th. $a \cdot y = 0$.

$(n-1)$ lin. indep. vectors y_i (over $\mathbb{Z}_2$) s.th. $a \cdot y_i = 0$

allow to determine a (solve lin. eq. – e.g. Gaussian elimination).

Space of lin. dep. vectors of k vectors grows as 2^k

$\Rightarrow$ $O(1)$ chance to find randomly a lin. indep. vector

$\Rightarrow$ $O(n)$ random y are enough

$\Rightarrow$ $O(n)$ oracle queries are enough (on average)

Classical: 2^{cn} queries
Quantum: $c \cdot n$ queries

} exponential speed-up

(in terms of oracle queries)

Notes: • We don't have to measure B — we never use the outcome! (But: Derivation easier this way!)

• $H^{\otimes n} \cong$ (discrete) Fourier transform over $\mathbb{Z}_2^{\times n}$

$\rightarrow$ period finding via Fourier transform

3. The quantum Fourier transform, period finding, and Shor's factoring algorithm

Can we go beyond Fourier trafo on $\mathbb{H}_2$ (to $\mathbb{H}_N$, for $N \sim 2^n$)?

- What is the right transformation?
- Can it be implemented efficiently?
- What is it good for?

Further reading:
A. Ekert and R. Jozsa,
Quantum computation and Shor's factoring algorithm.
Rev. Mod. Phys **68**, 733 (1996)
https:///doi.org/10.1103/RevModPhys.68.733

a) The Quantum Fourier Transform

Discrete Fourier trafo (FT) on $\mathbb{C}^N$:

$$x = (x_0, \dots, x_{N-1}) \in \mathbb{C}^N$$

$$y = (y_0, \dots, y_{N-1}) \in \mathbb{C}^N$$

FT: $F: x \longmapsto y$ s.t. $y_k = \frac{1}{\sqrt{N}} \sum_{j=0}^{N-1} x_j \, e^{2\pi i jk/N}$

Definition QFT

$$|j\rangle \longmapsto \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} e^{2\pi i jk/N} |k\rangle$$

Observe:

$$\sum_j x_j |j\rangle \xrightarrow{\text{QFT}} \sum_{jk} x_j \, e^{2\pi i jk/N} |k\rangle = \sum y_k |k\rangle$$

i.e.: QFT acts as discrete FT on amplitudes!

Computational cost of classical FT:

- $O(N^2)$ operations.
- $N \sim 2^n \Rightarrow$ exponential in # of bits in N.
- Fast FT (FFT): only $O(N \log N)$, but still exponential!
- $O(N)$ is lower bound: minimal time to even just output y_k!

Will see: QFT can be implemented on a quantum state in $O(n^2)$ steps

$\rightarrow$ exponential speedup!

(But only useful if input is given as q. state!)

Step I: Rewrite QFT in binary

- Consider case $N = 2^n$.
- Write j etc. in binary:

$$j = j_1 j_2 j_3 \cdots j_n = j_1 \cdot 2^{n-1} + j_2 2^{n-2} + \cdots + j_n 2^0$$

- "Decimal" point notation:

$$0.j_\ell j_{\ell+1} \cdots j_n = \frac{1}{2} j_\ell + \frac{1}{4} j_{\ell+1} + \cdots + \frac{1}{2^{n-\ell+1}} j_n$$

Then:

$$|j\rangle \longmapsto \frac{1}{2^{n/2}} \sum_{k=0}^{2^n-1} e^{2\pi i \, j \cdot \frac{k}{2^n}} |k\rangle \qquad \frac{k}{2^n} = 0.k_1 k_2 \cdots k_n$$

$$= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} e^{2\pi i j \left(\sum_{\ell=1}^{n} k_\ell 2^{-\ell} \right)} |k_1, \ldots, k_n\rangle$$

$$= \frac{1}{2^{n/2}} \sum_{k_1=0}^{1} \cdots \sum_{k_n=0}^{1} \left[\bigotimes_{\ell=1}^{n} \left(e^{2\pi i j k_\ell 2^{-\ell}} |k_\ell\rangle \right) \right]$$

$$= \bigotimes_{\ell=1}^{n} \left[\frac{1}{\sqrt{2}} \sum_{k_\ell=0}^{1} e^{2\pi i j k_\ell 2^{-\ell}} |k_\ell\rangle \right]$$

$$= \bigotimes_{\ell=1}^{n} \frac{1}{\sqrt{2}} \left[|0\rangle + e^{2\pi i j 2^{-\ell}} |1\rangle \right] = \dots$$

$$j \cdot 2^{-\ell} = \underbrace{j_1 j_2 \dots j_{n-\ell}}_{\text{integer}} \cdot j_{n-\ell+1} \dots j_n$$

$$e^{2\pi i (j \cdot 2^{-\ell})} = e^{2\pi i \cdot (\text{integer} + 0.j_{n-\ell+1} \dots j_n)}$$

$$= e^{2\pi i \cdot 0.j_{n-\ell+1} \dots j_n}$$

$$\dots = \frac{|0\rangle + e^{2\pi i\, 0.j_n} |1\rangle}{\sqrt{2}} \otimes \frac{|0\rangle + e^{2\pi i\, 0.j_{n-1} j_n} |1\rangle}{\sqrt{2}} \otimes \dots$$

$$\dots \otimes \frac{|0\rangle + e^{2\pi i\, 0.j_1 j_2 \dots j_n} |1\rangle}{\sqrt{2}}$$

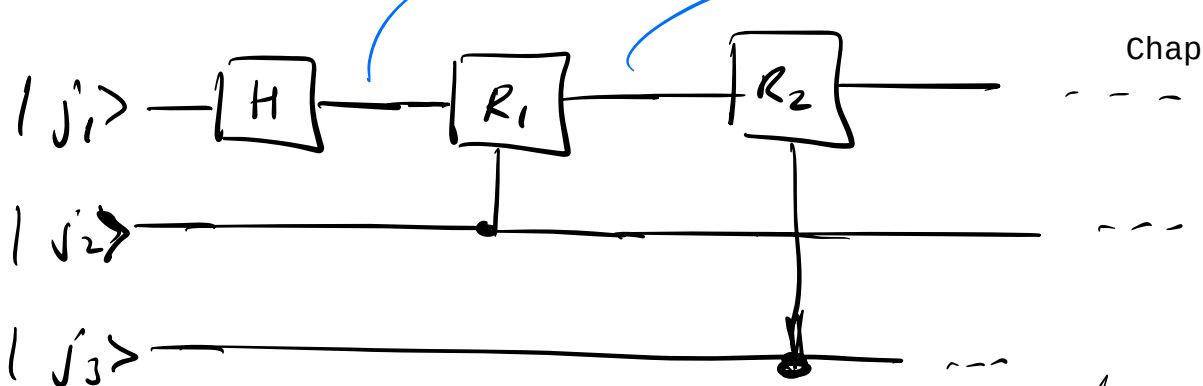
Step II: Implement this as a circuit.

Consider first only rightmost term:

$$\frac{|0\rangle + e^{2\pi i\, 0.j_1 j_2 \dots j_n} |1\rangle}{\sqrt{2}} = \frac{|0\rangle + e^{2\pi i\, j_1/2}\, e^{2\pi i\, j_2/4}\, e^{2\pi i\, j_3/8} \dots |1\rangle}{\sqrt{2}}$$

$(-1)^{j_1}$

$$|0\rangle + e^{2\pi i j_1/2} |1\rangle \qquad |0\rangle + e^{2\pi i\, j_1/2}\, e^{2\pi i\, j_2/4} |1\rangle$$



$$R_d = \begin{pmatrix} 1 & & \\ & \ddots & \\ & & e^{2\pi i \cdot 2^{-(d+1)}} \end{pmatrix}$$

Action of gates:

$$H: |j_1\rangle \longmapsto |0\rangle + e^{2\pi i\, 0.j_1} |1\rangle$$

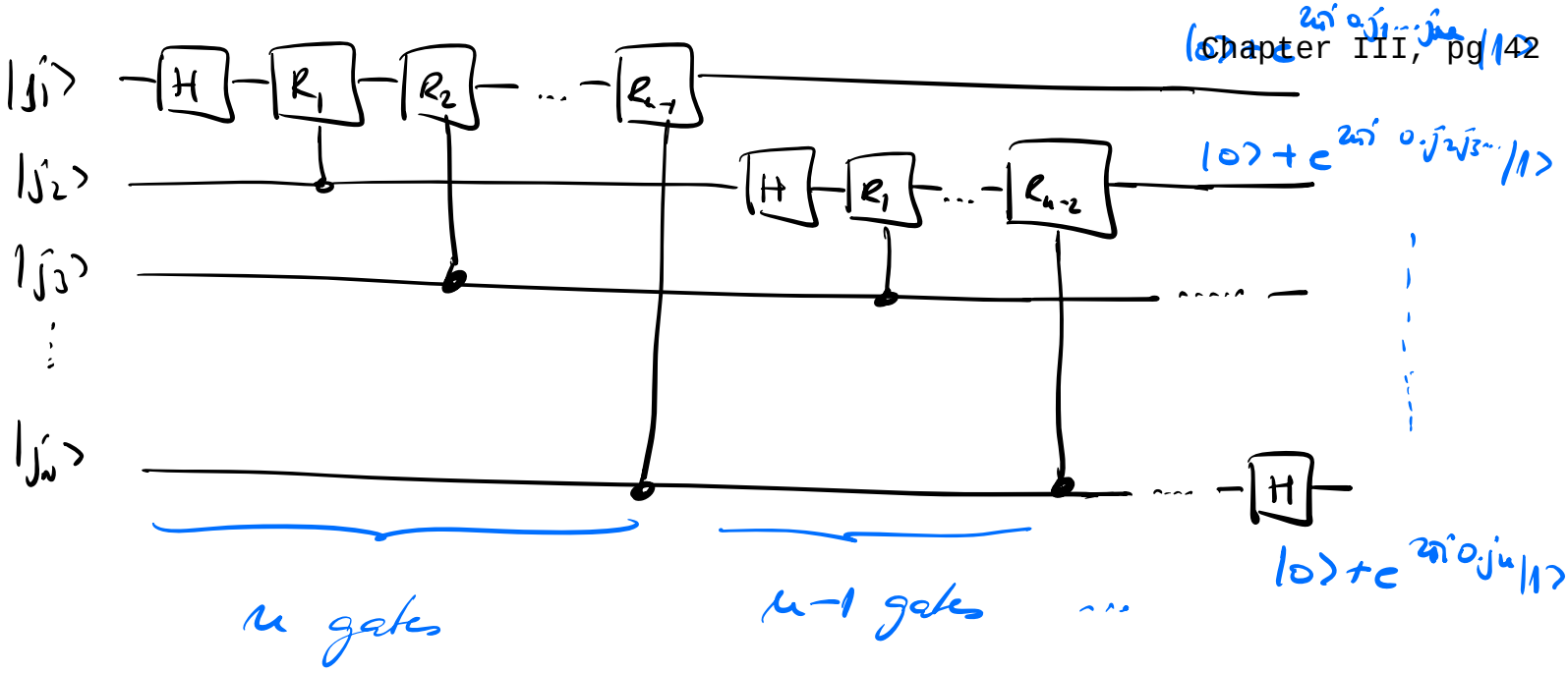
$$C\text{-}R_1: \left(|0\rangle + e^{2\pi i\, 0.j_1} |1\rangle\right)|j_2\rangle \longmapsto \left(|0\rangle + e^{2\pi i\, 0.j_1 j_2} |1\rangle\right)|j_2\rangle$$

$$C\text{-}R_2: \left(|0\rangle + e^{2\pi i\, 0.j_1 j_2}\right)|j_2\rangle|j_3\rangle \longmapsto \left(|0\rangle + e^{2\pi i\, 0.j_1 j_2 j_3} |1\rangle\right)|j_2\rangle|j_3\rangle$$

$\vdots$ and so on.

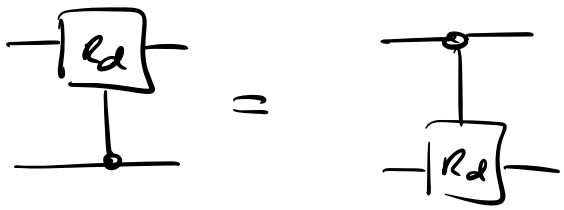
$\longrightarrow$ Outputs the n-th qubit of the QFT on 1st qubit.

Continue on this vein:



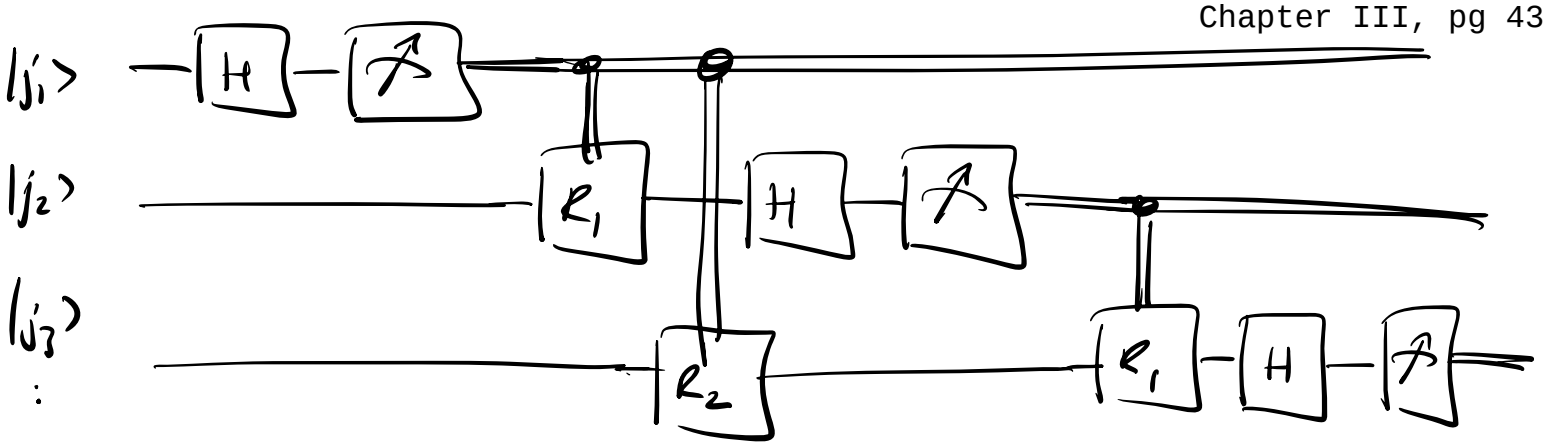
Gate count: $\frac{n(n+1)}{2} = O(n^2)$ gates!

Notes:

- Output qubits in reverse order (can re-order if needed: $n/2$ swaps).
-  ⇒ can flip C-R_d gate

Then, upper line acts as control in comp. basis.

⇒ If we measure directly after QFT in comp. basis, we can measure before the C-R_d gates & control them classically:



Only one-qubit gates needed (!!)

("Where is the quantum-ness?")

b) Period finding

Application of QFT: find period of a function?

(cf Shor's algorithm)

Consider a periodic function $f: \mathbb{N} \to \{0, \dots, n-1\}$, such that $\exists r > 0$ with

$$f(x) = f(x+r), \text{ and } f(x) \neq f(y) \text{ otherwise.}$$

On a computer, we can only compute f on a truncated input,

$$f: \underbrace{\{0, \dots, N-1\}}_{= \{0,1\}^m} \longrightarrow \underbrace{\{0, \dots, n-1\}}_{= \{0,1\}^m}$$

(In particular, the periodicity of f is broken across the boundary, if we think of $f(x+r) \equiv f((x+r) \bmod N)$)

Can we find r better than classically?

(i.e., with much less than $\sim r$ queries to f)

Choose n such that $2^n \gg r$

(will make this specific later. Goal: imperfection at most negligible.

Note: Since we do not know r, we need to know some upper bound on r – e.g., we can use that $r \leq N$.

Implement U_f on quantum computer as before:

$$U_f : |x\rangle_A |y\rangle_B \longmapsto |x\rangle_A |y \oplus f(x)\rangle_B$$

Algorithm:

(1) Hadamard on A, then U_f:

$$\frac{1}{2^{n/2}} \sum |x\rangle_A |0\rangle_B \xrightarrow{U_f} \frac{1}{2^{n/2}} \sum |x\rangle_A |f(x)\rangle_B$$

(2) Measure B register. For result $|f(x_0)\rangle_B$,

A collapses to

$$\frac{1}{\sqrt{k_0}} \sum_{k=0}^{k_0-1} |x_0 + kr\rangle$$

- here, $0 \leq x_0 < r$, and $\frac{2^4}{r} - 1 < k_0 \leq \frac{2^4}{r}$.

(3) Apply QFT:

$$\longmapsto \frac{1}{2^{4/2}\sqrt{k_0}} \sum_{k=0}^{k_0-1} \sum_{\ell=0}^{2^4-1} e^{2\pi i (x_0 + kr)\ell/2^4} |\ell\rangle_A$$

$$= \sum_{\ell=0}^{2^4-1} e^{2\pi i x_0 \ell/2^4} \underbrace{\sum_{k=0}^{k_0-1} \frac{1}{2^{4/2}\sqrt{k_0}} e^{2\pi i k r \ell/2^4}}_{=: \hat{a}_\ell} |\ell\rangle_A$$

$$=: a_\ell$$

(4) Measure in computational basis:

$|\hat{a}_\ell|^2$: probability to obtain outcome ℓ

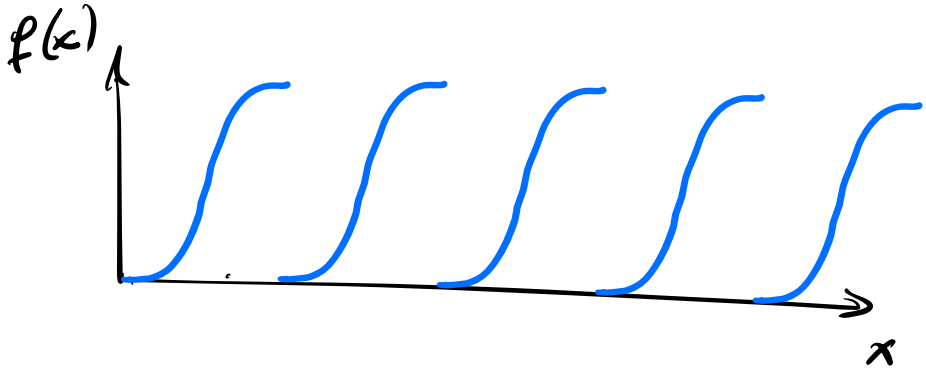
Intuitively: $\hat{a}_\ell \propto \sum_k e^{2\pi i k \left(r\ell/2^4\right)}$

peaked around points ℓ where $\frac{r\ell}{2^4}$ is close to an integer!

(→ will quantify this in a moment!)

Intuitive picture:

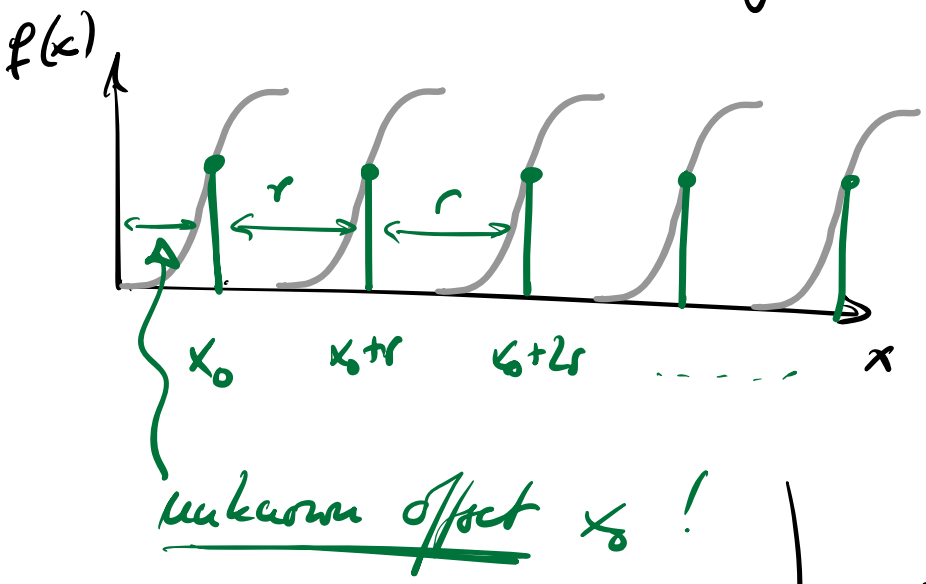
(General features of Fourier transforms — valley quantum!)



periodic function



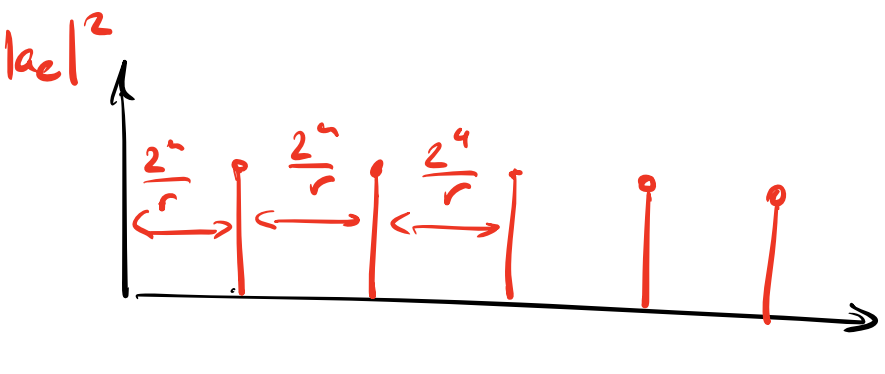
after meas. of $B \longrightarrow x_0$



unknown offset x_0 !



Fourier trafo



unknown offset: absorbed in phase of a_e!

$\Rightarrow$ can determine multiple of $\frac{2^n}{r}$ by measuring ℓ (How to get r? Later!)

Detailed analysis of $|a_\ell|^2$:

How much total weight is on all $|a_\ell|^2$ with

$$\ell = \frac{2^n}{r} \cdot s + \delta_s \; ; \quad \delta_s \in \left(-\tfrac{1}{2}, \tfrac{1}{2}\right] ; \quad s = 0, \dots, r-1$$

(i.e. only those ℓ which are closest to $\frac{2^n}{r} \cdot s$)

Then, $\hat{a}_\ell = \frac{1}{2^{n/2}\sqrt{k_0}} \sum_{k=0}^{k_0-1} e^{2\pi i k \underbrace{\left(s + \frac{r}{2^n}\delta_s\right)}_{\equiv r\ell/2^n}}$

$$= \frac{1}{2^{n/2}\sqrt{k_0}} \; \frac{e^{2\pi i \frac{r}{2^n}\delta_s k_0} - 1}{e^{2\pi i \frac{r}{2^n}\delta_s} - 1}$$

... since $\frac{2^n}{r} - 1 < k_0 \leq \frac{2^n}{r}$, and $r \ll 2^n$:

$$\frac{k_0 r}{2^n} = 1 - \varepsilon \; , \quad 0 \leq \varepsilon < \frac{r}{2^n} \ll 1.$$

$$= \frac{1}{2^{4/2}\sqrt{k_0}} \; \frac{e^{2\pi i \delta_S (1-\varepsilon)} - 1}{e^{2\pi i \frac{r}{2^u} \delta_S} - 1}$$

$|\sin x| \geq \frac{|x|}{\pi/2}$ in relev. interval

$$\Rightarrow \quad |q_e|^2 = \frac{1}{2^u k_0} \left(\frac{|\sin(\pi \delta_S (1-\varepsilon))|}{|\sin\left(\frac{\pi r}{2^u} \delta_S\right)|} \right)^2$$

$|\sin x| \leq |x|$

$$\geq \frac{1}{2^u k_0} \; \frac{\frac{\pi^2 \delta_S^2 (1-\varepsilon)^2}{\pi^2/4}}{\frac{\pi^2 r^2}{(2^u)^2} \delta_S^2}$$

$$= \frac{4}{\pi^2} \frac{1}{r} \; \frac{(1-\varepsilon)^2}{\frac{k_0 r}{2^u}}$$

$\geq 1-\varepsilon$

$$= \frac{4}{\pi^2} \frac{1}{r} (1-\varepsilon) \approx \frac{4}{\pi^2} \frac{1}{r}$$

(can be easily made more quantitative, using $\varepsilon < \frac{r}{2^u}$!)

since $s = 0, \ldots, r-1$: Total probability that

$\left| \ell - \frac{2^n}{r} s \right| \leq \frac{1}{2}$ for one such s: $p \geq \frac{4}{\pi^2} \approx 0.41$

With sufficiently high probability — we will see that we can <u>check</u> success and thus repeat until we succeed! — we obtain an ℓ s.th. $\ell = \frac{2^n}{r} s + \delta_s$, and thus,

$$\frac{\ell}{2^n} \approx \frac{s}{r},$$

where s is chosen uniformly at random.

If we choose $r \ll 2^n$ suitably, there is only <u>one</u> such ratio $\frac{s}{r}$ with $\left| \ell - \frac{2^n}{r} s \right| \leq 1/2$, and it can be found efficiently. (See further reading.)

Specifically, it suffices to choose $N = 2^n = (2^m)^2 = M^2$, i.e. $n = 2m$, and since $M \geq r$: $2^n \gg 2^{n/2} \geq r$.

If s and r are co-prime, i.e. $\gcd(r,s)=1$, we can infer r from $\frac{s}{r}$. This happens with probability at least $p(\gcd(s,r)=1) \geq 1/\log r \geq \frac{2}{\log 2} \cdot \frac{1}{n}$.

(at least all primes $2 \leq s < r$ are good, and density of primes goes as $1/\log r$.)

$\Rightarrow$ with $O(n)$ iterations, we find a s coprime with r.

Once we have used this to obtain a guess for r, we can test whether $f(x) = f(x+r)$, and repeat until success!

$\Rightarrow$ Efficient algorithm for period finding.

$\sim O(n)$ applications of f required!

c) Application: Factoring Algorithm

Factoring: Given $N \in \mathbb{N}$ (not prime), find

$$f \in \mathbb{N},\ f \neq 1,\ \text{such that}\ f \mid N.$$

("f divides N")

(Note: Primality of N can be checked efficiently.)

This can be solved efficiently if we have an efficient method for period finding!

Sketch of algorithm:

(1) Select a random a, $2 \leq a < N$.

If $\gcd(a, N) > 1 \Rightarrow$ done, $f = \gcd(a, N)$!

↑ eff. computable!

Thus: Assume $\gcd(a, N) = 1$.

(2) Denote by r the smallest $x > 0$ such that

$$a^x \bmod N = 1.$$

— that is, the period of

$$f_{N,a}(x) := a^x \bmod N$$

r is called the <u>order of a mod N</u>.

(<u>Note</u>: Some $z > 1$ s.t. $a^z \bmod N = 1$ must exist since

$$\exists\, x, y \in \{1, \dots, N\}: \; a^x \equiv a^y \bmod N \quad \text{(counting possibilities)}$$

$$\Rightarrow a^x \left(1 - a^{y-x}\right) \equiv 0 \bmod N$$

$$\Rightarrow N \mid \left(a^x \left(1 - a^{y-x}\right)\right)$$

$$\overset{\gcd(a,N)=1}{\Longrightarrow} N \mid \left(1 - a^{y-x}\right)$$

$$\Rightarrow a^{y-x} \equiv 1 \bmod N \quad \square \;)$$

Recall: "Efficient" means "polynomial in # of digits of N"

Furthermore, $f_{N,a}(x)$ can be <u>computed efficiently</u>:

Why $x = x_{m-1} 2^{m-1} + x_{m-2} 2^{m-2} + \dots$,

$$a^x \bmod N = \underbrace{\left(a^{(2^{m-1})}\right)^{x_{m-1}}}_{\text{eff. computable via repeated squaring mod } N:} \cdot \left(a^{(2^{m-2})}\right)^{x_{m-2}} \cdot \ldots \bmod N$$

$$a \mapsto a^2 \bmod N \mapsto \underbrace{a^4 \bmod N}_{\equiv (a^2 \bmod N)^2 \bmod N} \mapsto \ldots,$$

by doing "mod N" in each step the numbers don't require an exp. number of digits:

$O(n)$ multiplications of n-digit numbers.

$\Rightarrow$ r can be <u>found efficiently</u> with a <u>quantum computer</u>!

(3) Assume for now <u>r even</u>:

$$a^r \bmod N = 1$$

$$\Leftrightarrow N \mid (a^r - 1),$$

$$\Leftrightarrow N \mid (a^{r/2} + 1)(a^{r/2} - 1)$$

However, we also know that $N \nmid (a^{r/2} - 1)$,

since otherwise $a^{r/2} \bmod N = 1$ ↯ (does not divide)

$\Rightarrow$ either $N \mid a^{r/2} + 1$

or N has non-trivial common factors with both $a^{r/2} \pm 1$.

$\Rightarrow 1 \neq f := \gcd(N, a^{r/2}+1) \mid N$

$\Rightarrow$ found a non-trivial factor f of N!

$\Rightarrow$ Algorithm will succeed as long as

(i) r even

(ii) $N \nmid (a^{r/2}+1)$

This can be shown to happen with prob. $\geq 1/2$ for a random choice of a (see further reading)

- unless either N is even

(can be checked efficiently),

or $N = p^k$, p prime

(can also be checked efficiently by factoring roots; there are only $O(\log(N))$ roots which one has to check!)

- and in both cases, this gives a non-trivial factor!

$\Rightarrow$ efficient Quantum Algorithm for Factoring.

"Shor's algorithm"